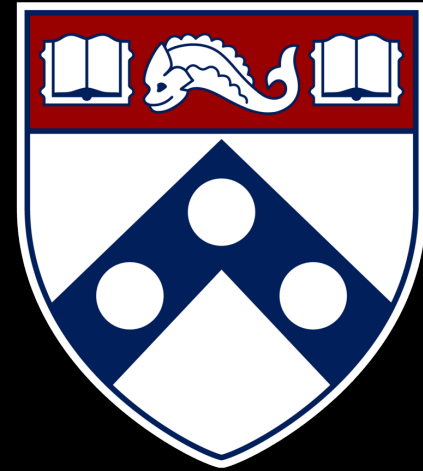


Empirical Security & Privacy, for Humans



UPenn CIS 7000-010
9/30/2025



Evaluating fuzz testing

Readings

Session 10D: VulnDet 2 + Side Channels 2

CCS'18, October 15–19, 2018, Toronto, ON, Canada

Evaluating Fuzz Testing

George Klees, Andrew Ruef,
Benji Cooper
University of Maryland

Shiyi Wei
University of Texas at Dallas

Michael Hicks
University of Maryland

ABSTRACT

Fuzz testing has enjoyed great success at discovering security critical bugs in real software. Recently, researchers have devoted significant effort to devising new fuzzing techniques, strategies, and algorithms. Such new ideas are primarily evaluated experimentally so an important question is: What experimental setup is needed to produce trustworthy results? We surveyed the recent research literature and assessed the experimental evaluations carried out by 32 fuzzing papers. We found problems in every evaluation we considered. We then performed our own extensive experimental evaluation using an existing fuzzer. Our results showed that the general problems we found in existing experimental evaluations can indeed translate to actual wrong or misleading assessments. We conclude with some guidelines that we hope will help improve experimental evaluations of fuzz testing algorithms, making reported results more robust.

CCS CONCEPTS

• Security and privacy → Software and application security;

KEYWORDS

fuzzing, evaluation, security

ACM Reference Format:

George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*, October 15–19, 2018, Toronto, ON, Canada. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3243734.3243804>

1 INTRODUCTION

A *fuzz tester* (or *fuzzer*) is a tool that iteratively and randomly generates inputs with which it tests a target program. Despite appearing “naive” when compared to more sophisticated tools involving SMT solvers, symbolic execution, and static analysis, fuzzers are sur-

Why do we think fuzzers work? While inspiration for new ideas may be drawn from mathematical analysis, fuzzers are primarily evaluated experimentally. When a researcher develops a new fuzzer algorithm (call it *A*), they must empirically demonstrate that it provides an advantage over the status quo. To do this, they must choose:

- a compelling *baseline* fuzzer *B* to compare against;
- a sample of target programs—the *benchmark suite*;
- a *performance metric* to measure when *A* and *B* are run on the benchmark suite; ideally, this is the number of (possibly exploitable) bugs identified by crashing inputs;
- a meaningful set of *configuration parameters*, e.g., the *seed file* (or files) to start fuzzing with, and the *timeout* (i.e., the duration) of a fuzzing run.

An evaluation should also account for the fundamentally random nature of fuzzing: Each fuzzing run on a target program may produce different results than the last due to the use of randomness. As such, an evaluation should measure *sufficiently many trials* to sample the overall distribution that represents the fuzzer’s performance, using a *statistical test* [38] to determine that *A*’s measured improvement over *B* is real, rather than due to chance.

Failure to perform one of these steps, or failing to follow recommended practice when carrying it out, could lead to misleading or incorrect conclusions. Such conclusions waste time for practitioners, who might profit more from using alternative methods or configurations. They also waste the time of researchers, who make overly strong assumptions based on an arbitrary tuning of evaluation parameters.

We examined 32 recently published papers on fuzz testing (see Table 1) located by perusing top-conference proceedings and other quality venues, and studied their experimental evaluations. We found that no fuzz testing evaluation carries out all of the above steps properly (though some get close). This is bad news in theory, and after carrying out more than 50,000 CPU hours of experiments,

SoK: Prudent Evaluation Practices for Fuzzing

Moritz Schloegel¹, Nils Bars¹, Nico Schiller¹, Lukas Bernhard¹, Tobias Scharnowski¹, Addison Crump¹, Arash Ale-Ebrahim¹, Nicolai Bissantz², Marius Muench³, Thorsten Holz¹

¹CISPA Helmholtz Center for Information Security, {first.lastname}@cispa.de

²Ruhr University Bochum, nicolai.bissantz@ruhr-uni-bochum.de

³University of Birmingham, m.muench@bham.ac.uk

Abstract—Fuzzing has proven to be a highly effective approach to uncover software bugs over the past decade. After AFL popularized the groundbreaking concept of lightweight coverage feedback, the field of fuzzing has seen a vast amount of scientific work proposing new techniques, improving methodological aspects of existing strategies, or porting existing methods to new domains. All such work must demonstrate its merit by showing its applicability to a problem, measuring its performance, and often showing its superiority over existing works in a thorough, empirical evaluation. Yet, fuzzing is highly sensitive to its target, environment, and circumstances, e.g., randomness in the testing process. After all, relying on randomness is one of the core principles of fuzzing, governing many aspects of a fuzzer’s behavior. Combined with the often highly difficult to control environment, the *reproducibility* of experiments is a crucial concern and requires a prudent evaluation setup. To address these threats to validity, several works, most notably *Evaluating Fuzz Testing* by Klees et al., have outlined how a carefully designed evaluation setup should be implemented, but it remains unknown to what extent their recommendations have been adopted in practice.

In this work, we systematically analyze the evaluation of 150 fuzzing papers published at the top venues between 2018 and 2023. We study how existing guidelines are implemented and observe potential shortcomings and pitfalls. We

1. Introduction

Fuzzing, a portmanteau of “fuzz testing”, has gained much attention in recent years, and the method has proven to be highly successful in uncovering many types of faults in software systems. Companies such as Meta, Google, and Oracle have invested significant resources in this technology and use it to test their products. Large software projects such as web browsers or the Linux kernel incorporate fuzzing into their development cycle, and Google is running an extensive and continuous fuzzing campaign for more than 1,200 open-source projects via OSS-Fuzz [62]. Beyond the wide acceptance in the industry, a large number of academic papers have proposed numerous improvements and novel techniques to enhance fuzzing further. More specifically, we found that, over the past six years, more than 280 papers on fuzzing have been published in the top computer security and software engineering venues.

A cornerstone of fuzzing research, and science in general, is that other researchers can critically assess the correctness of scientific results. To this end, the research results must be *reproducible*, meaning that another group should be able to obtain the same results using the same experimental setup, often by using a research artifact provided by the authors [8]. Reproducibility is paramount for other researchers to understand, trust, and build on the research results.

Readings

Session 10D: VulnDet 2 + Side Channels 2

CCS'18, October 15-19, 2018, Toronto, ON, Canada

Evaluating Fuzz Testing

George Klees, Andrew Ruef,
Benji Cooper
University of Maryland

Shiyi Wei
University of Texas at Dallas

Michael Hicks
University of Maryland

ABSTRACT

Fuzz testing has enjoyed great success at discovering security critical bugs in real software. Recently, researchers have devoted significant effort to devising new fuzzing techniques, strategies, and algorithms. Such new ideas are primarily evaluated experimentally so an important question is: What experimental setup is needed to produce trustworthy results? We surveyed the recent research literature and assessed the experimental evaluations carried out by 32 fuzzing papers. We found problems in every evaluation we considered. We then performed our own extensive experimental evaluation using an existing fuzzer. Our results showed that the general problems we found in existing experimental evaluations can indeed translate to actual wrong or misleading assessments. We conclude with some guidelines that we hope will help improve experimental evaluations of fuzz testing algorithms, making reported results more robust.

CCS CONCEPTS

• Security and privacy → Software and application security;

KEYWORDS

fuzzing, evaluation, security

ACM Reference Format:

George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*, October 15–19, 2018, Toronto, ON, Canada. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3243734.3243804>

1 INTRODUCTION

A *fuzz tester* (or *fuzzer*) is a tool that iteratively and randomly generates inputs with which it tests a target program. Despite appearing “naive” when compared to more sophisticated tools involving SMT solvers, symbolic execution, and static analysis, fuzzers are sur-

Why do we think fuzzers work? While inspiration for new ideas may be drawn from mathematical analysis, fuzzers are primarily evaluated experimentally. When a researcher develops a new fuzzer algorithm (call it *A*), they must empirically demonstrate that it provides an advantage over the status quo. To do this, they must choose:

- a compelling *baseline* fuzzer *B* to compare against;
- a sample of target programs—the *benchmark suite*;
- a *performance metric* to measure when *A* and *B* are run on the benchmark suite; ideally, this is the number of (possibly exploitable) bugs identified by crashing inputs;
- a meaningful set of *configuration parameters*, e.g., the *seed file* (or files) to start fuzzing with, and the *timeout* (i.e., the duration) of a fuzzing run.

An evaluation should also account for the fundamentally random nature of fuzzing: Each fuzzing run on a target program may produce different results than the last due to the use of randomness. As such, an evaluation should measure *sufficiently many trials* to sample the overall distribution that represents the fuzzer’s performance, using a *statistical test* [38] to determine that *A*’s measured improvement over *B* is real, rather than due to chance.

Failure to perform one of these steps, or failing to follow recommended practice when carrying it out, could lead to misleading or incorrect conclusions. Such conclusions waste time for practitioners, who might profit more from using alternative methods or configurations. They also waste the time of researchers, who make overly strong assumptions based on an arbitrary tuning of evaluation parameters.

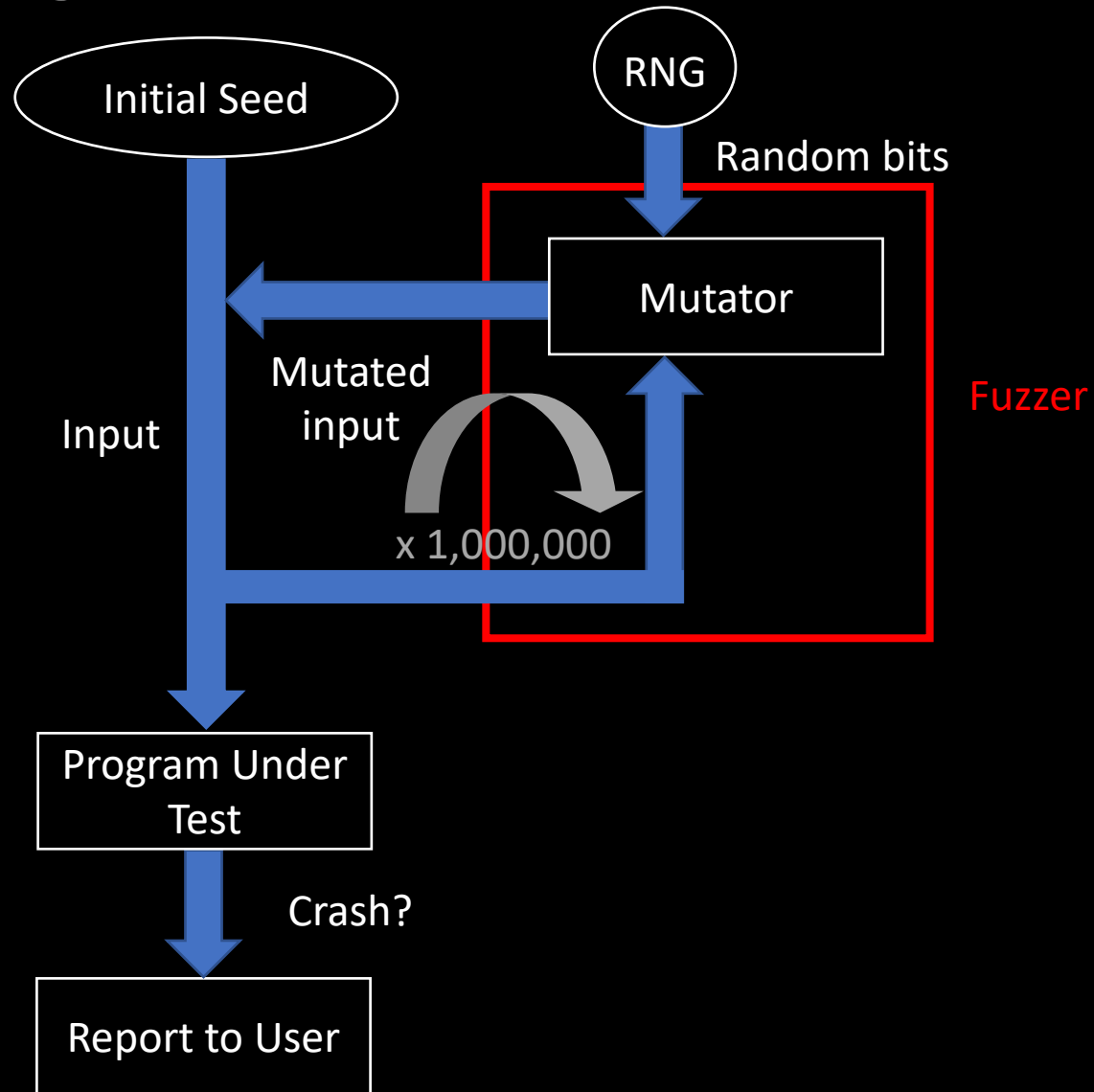
We examined 32 recently published papers on fuzz testing (see Table 1) located by perusing top-conference proceedings and other quality venues, and studied their experimental evaluations. We found that no fuzz testing evaluation carries out all of the above steps properly (though some get close). This is bad news in theory, and -after carrying out more than 50,000 CPU hours of experiments-

Fuzzing

- A kind of **testing** based on **random input generation**
- **Goal:** make sure certain **bad things don't happen, no matter what**
 - **Crashes, thrown exceptions, non-termination**
 - All of these things can be the foundation of security vulnerabilities

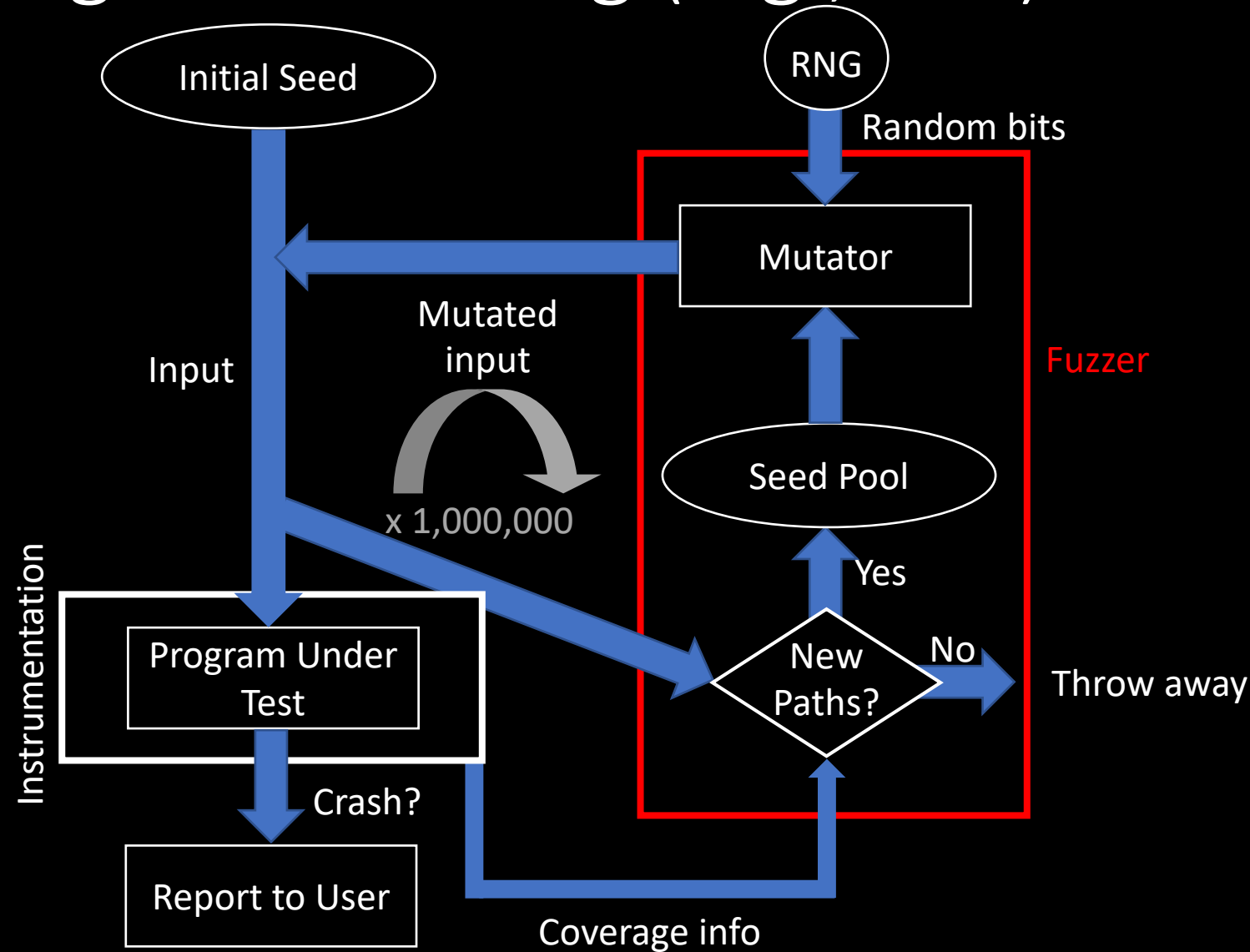
Blackbox fuzzing

Repeatedly
mutate an
initial seed



Coverage-guided fuzzing (e.g., AFL)

Only
mutate
seeds that
induce
new
execution
paths



american fuzzy lop 0.47b (readpng)

process timing

run time : 0 days, 0 hrs, 4 min, 43 sec
last new path : 0 days, 0 hrs, 0 min, 26 sec
last uniq crash : none seen yet
last uniq hang : 0 days, 0 hrs, 1 min, 51 sec

cycle progress

now processing : 38 (19.49%)
paths timed out : 0 (0.00%)

stage progress

now trying : interest 32/8
stage execs : 0/9990 (0.00%)
total execs : 654k
exec speed : 2306/sec

fuzzing strategy yields

bit flips : 88/14.4k, 6/14.4k, 6/14.4k
byte flips : 0/1804, 0/1786, 1/1750
arithmetics : 31/126k, 3/45.6k, 1/17.8k
known ints : 1/15.8k, 4/65.8k, 6/78.2k
havoc : 34/254k, 0/0
trim : 2876 B/931 (61.45% gain)

overall results

cycles done : 0
total paths : 195
uniq crashes : 0
uniq hangs : 1

map coverage

map density : 1217 (7.43%)
count coverage : 2.55 bits/tuple

findings in depth

favorable paths : 128 (65.64%)
new edges on : 85 (43.59%)
total crashes : 0 (0 unique)
total hangs : 1 (1 unique)

path geometry

levels : 3
pending : 178
pend fav : 114
imported : 0
variable : 0
latent : 0

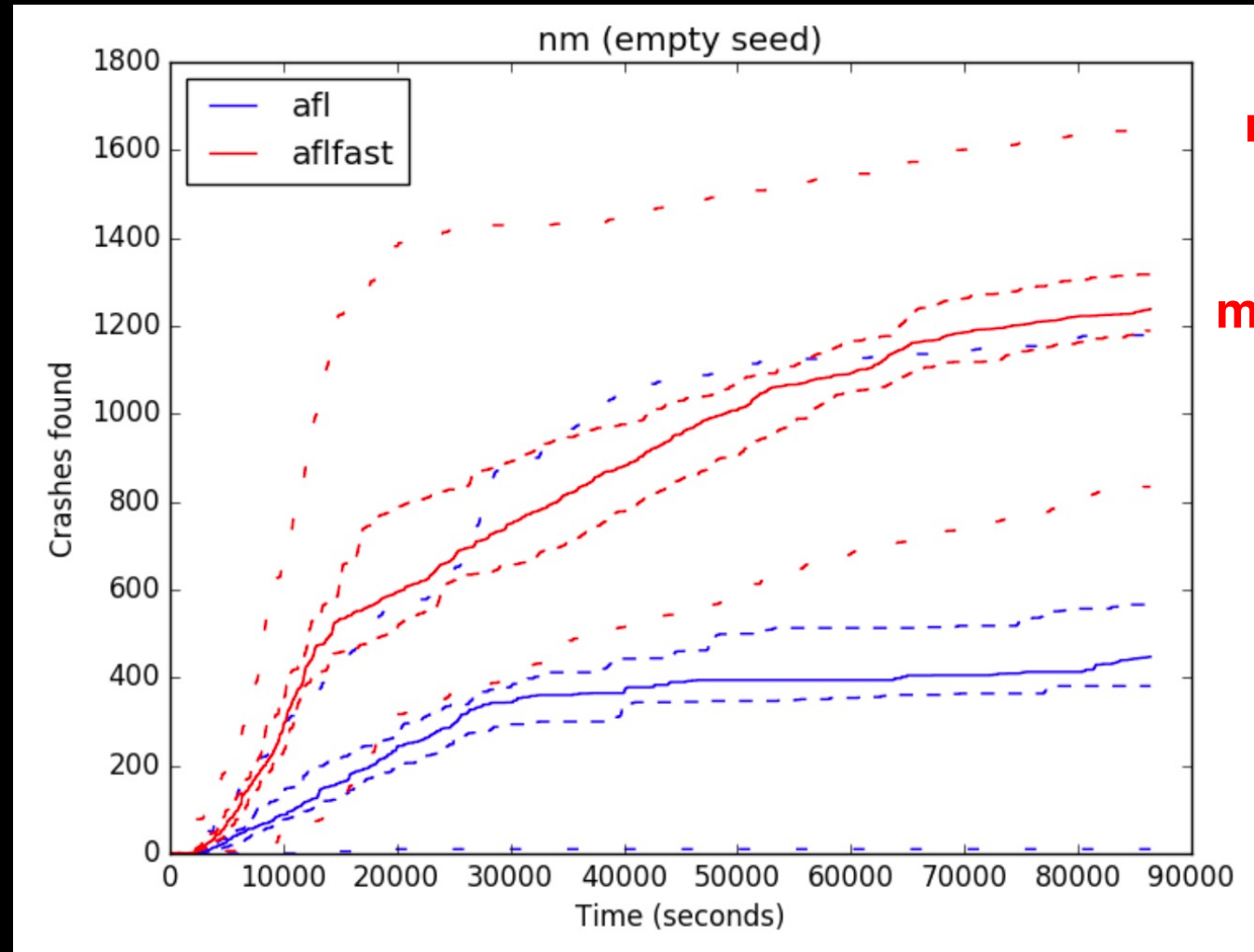
Fuzzing Evaluation Recipe (for new fuzzer A)

- A compelling **baseline** fuzzer B to compare against
- A **sample of target programs** (benchmark suite)
 - Representative of larger population
- A **performance metric**
 - Ideally, the number of bugs found (else a proxy)
- A meaningful set of configuration parameters
 - Notably, justifiable **seed file(s)**, **timeout**
- A sufficient **number of trials** to judge performance
 - Comparison with baseline using a **statistical test**

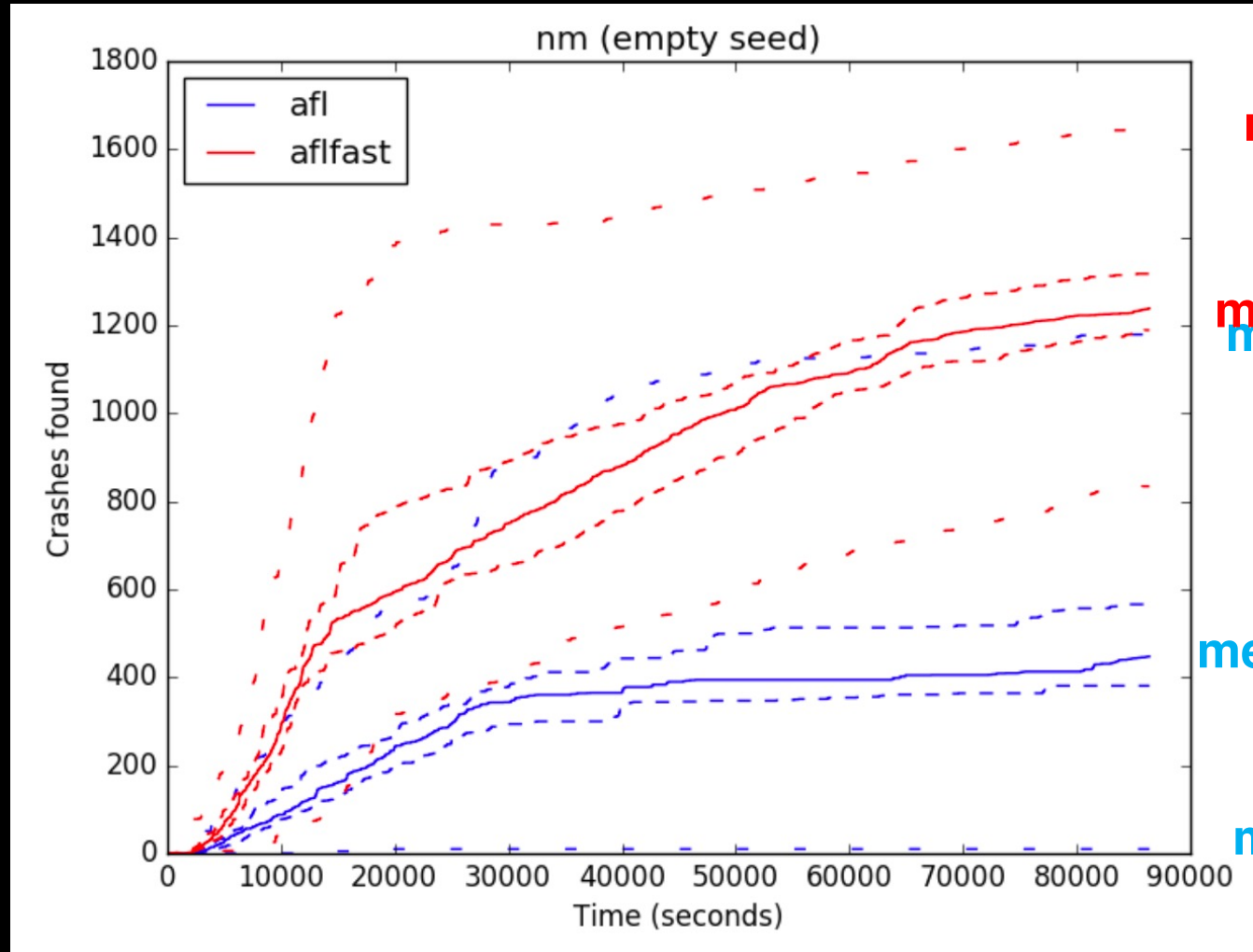
Summary of Results

- **Less than 1/2 of 32 papers we looked at did multiple runs**
 - Fewer still consider variance across runs
 - And yet fuzzer *performance can vary substantially from run to run*

Performance Plot



Performance Plot



max

95%

median

95%

max

min

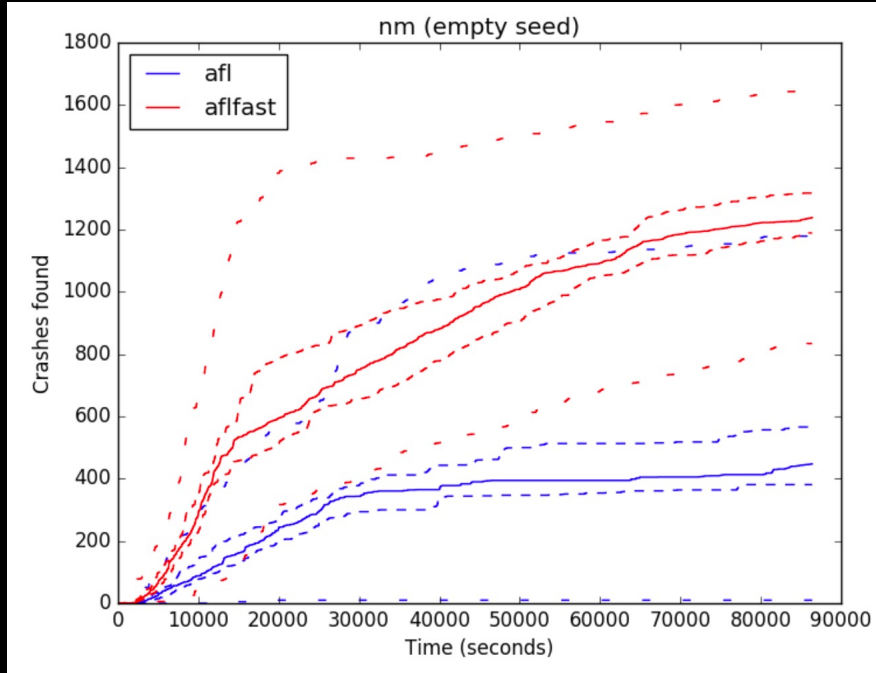
95%

median

95%

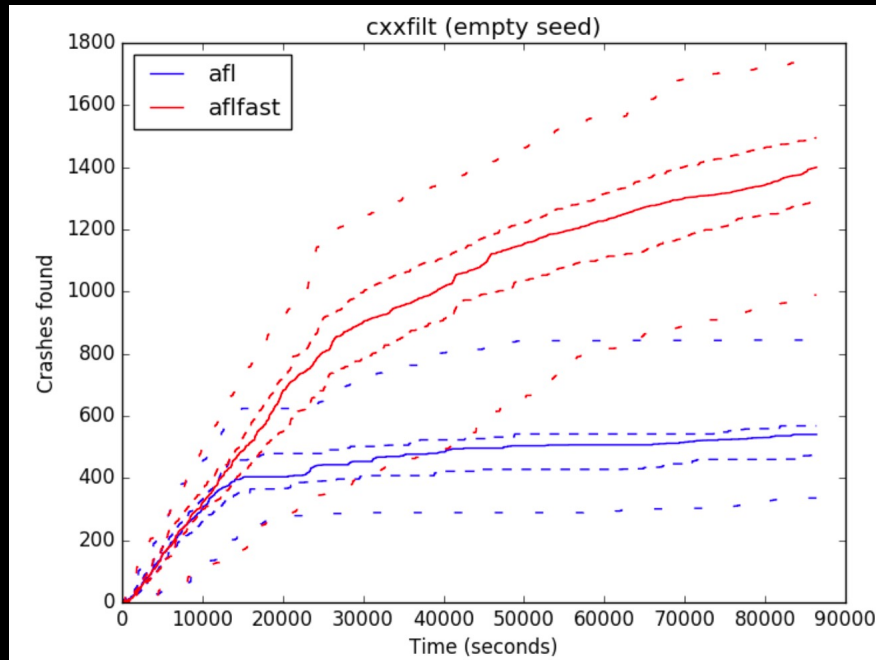
min

Statistically Significant



$p < 10^{-13}$

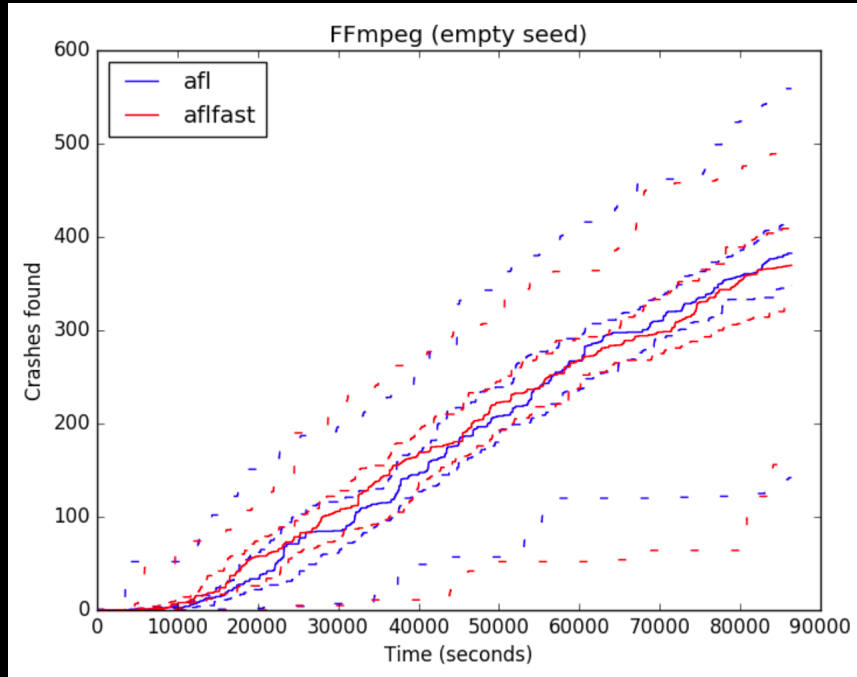
significant variance
in performance



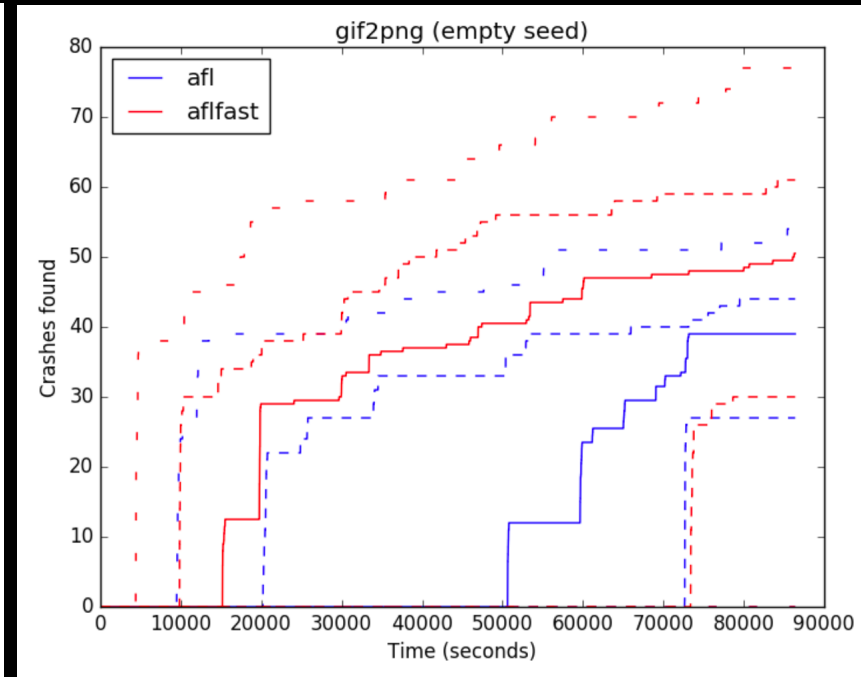
$p < 10^{-10}$

Higher median
clearly better

Statistically Insignificant



$p = 0.379$



$p = 0.0676$

Max **AFL** = 550

Min **AFLFast** = 150

Higher median
does *not* meet bar
for significance

Summary of Results

- **Less than 1/2 of 32 papers we looked at did multiple runs**
 - Fewer still consider variance across runs
 - And yet fuzzer *performance can vary substantially from run to run*
- Papers often choose a **small number of target programs**, with a **small common set**
 - And yet they target the same population
 - And *performance can vary substantially*
- **Few papers justify the choice of seeds or timeouts**
 - Yet *seeds strongly influence performance*,
 - And *trends can change over time*
- Many papers use **heuristics to relate crashing inputs to bugs**
 - Yet these *heuristics have not been evaluated*
 - We find that they **dramatically overcount bugs**

Unreliability of code coverage

- Coverage correlates with bug finding
- But coverage is no replacement: “More coverage” is not a reliable stand-in for “more bugs found”

On the Reliability of Coverage-Based Fuzzer Benchmarking

Marcel Böhme
MPI-SP, Germany
Monash University, Australia

László Szekeres
Google, USA

Jonathan Metzman
Google, USA

ABSTRACT

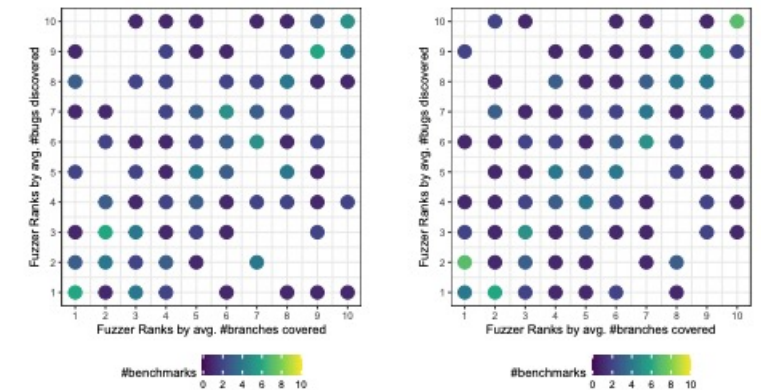
Given a program where none of our fuzzers finds any bugs, how do we know which fuzzer is better? In practice, we often look to code coverage as a proxy measure of fuzzer effectiveness and consider the fuzzer which achieves more coverage as the better one.

Indeed, evaluating 10 fuzzers for 23 hours on 24 programs, we find that a fuzzer that covers more code also finds more bugs. There is a *very strong correlation* between the coverage achieved and the number of bugs found by a fuzzer. Hence, it might seem reasonable to compare fuzzers in terms of coverage achieved, and from that derive empirical claims about a fuzzer’s superiority at finding bugs.

Curiously enough, however, we find *no strong agreement* on which fuzzer is superior if we compared multiple fuzzers in terms of coverage achieved instead of the number of bugs found. The fuzzer best at achieving coverage, may not be best at finding bugs.

ACM Reference Format:

Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the Relia-



(a) 1 hour fuzzing campaigns ($\rho = 0.38$). (b) 1 day fuzzing campaigns ($\rho = 0.49$).

Figure 1: Scatterplot of the ranks of 10 fuzzers applied to 24 programs for (a) 1 hour and (b) 23 hours, when ranking each fuzzer in terms of the avg. number of branches covered (x-axis) and in terms of the avg. number of bugs found (y-axis).

(Fuzzy) stack hashes

- Idea: **Identify bug** according to the **stack at the time of the crash** (return addresses)
 - Or: Limit attention to the top N frames (where N is between 3 and 5 in most papers)
- Rationale: **Faulting location highly indicative** of source of bug
 - **Stack provides useful context** (i.e., when faulting function given a input, only from certain caller)
 - But some “context” may be superfluous
 - Assume: frames closer to bug more relevant

False positives and false negatives

```
void f() { ... format(s1); ... }
void g() { ... format(s2); ... }
void format(char *s) {
    //bug: corrupt s
    prepare(s);
}
void prepare(char *s) {
    output(s);
}
void output(char *s) {
    //failure manifests
}
```

- With $N=3$, distinct calls to `format` from `f` and `g` will be conflated, properly
- But with $N=5$, calling `format` from `f` and `g` are made distinct
 - Overcounting
- With $N=2$, a bug in a different caller to `prepare` that corrupts its argument will be conflated with the `format` bug
 - Undercounting

AFL CMIN

- A crashing input is considered “unique” if either
 - the **coverage profile includes an edge (“tuple”) not seen** in any of the previous crashes
 - the profile is **missing a tuple always present in earlier faults**
- AFL calls this *CMIN*
 - Docs justify it by mentioning the issues with stack hashes
- **CMIN may also suffer from inflated counts** (false positives)
 - Many superfluously different paths to the same fault-point are treated as distinct

Assessing Heuristics: cxxfilt

#	Line 419	static struct demangle_component *d_sour	Line 423	static struct demangle_component *d_sour
423				
424		static long d_number (struct d_info *);		static long d_number (struct d_info *);
425				
426		static struct demangle_component *d_identifier (struct d_info *, int);		static struct demangle_component *d_identifier (struct d_info *, long);
427				
428		static struct demangle_component *d_operator_name (struct d_info *);		static struct demangle_component *d_operator_name (struct d_info *);
429				
#	Line 715	d_dump (struct demangle_component *dc, i	Line 719	d_dump (struct demangle_component *dc, i
719		case DEMANGLE_COMPONENT_FIXED_TYPE:		case DEMANGLE_COMPONENT_FIXED_TYPE:
720		printf ("fixed-point type, accum? %d, sat? %d\n",		printf ("fixed-point type, accum? %d, sat? %d\n",
721		dc->u.s_fixed.accum, dc->u.s_fixed.sat);		dc->u.s_fixed.accum, dc->u.s_fixed.sat);
722		d_dump (dc->u.s_fixed.length, indent + 2)		d_dump (dc->u.s_fixed.length, indent + 2);
723		break;		break;
724		case DEMANGLE_COMPONENT_ARGLIST:		case DEMANGLE_COMPONENT_ARGLIST:
725		printf ("argument list\n");		printf ("argument list\n");
#	Line 1656	d_number_component (struct d_info *di)	Line 1660	d_number_component (struct d_info *di)
1660		/* identifier ::= <(unqualified source code identifier)> */		/* identifier ::= <(unqualified source code identifier)> */
1661				
1662		static struct demangle_component *		static struct demangle_component *
1663		d_identifier (struct d_info *di, int len)		d_identifier (struct d_info *di, long len)
1664		{		{
1665		const char *name;		const char *name;
1666				
#	Line 1677	d_identifier (struct d_info *di, int len	Line 1681	d_identifier (struct d_info *di, int len
1681		/* Look for something which looks like a gcc encoding of an		/* Look for something which looks like a gcc encoding of an
1682		anonymous namespace, and replace it with a more user friendly		anonymous namespace, and replace it with a more user friendly
1683		name. */		name. */
1684		if (len >= (int) ANONYMOUS_NAMESPACE_PREFIX_LEN + 2		if (len >= (long) ANONYMOUS_NAMESPACE_PREFIX_LEN + 2
1685		&& memcmp (name, ANONYMOUS_NAMESPACE_PREFIX,		&& memcmp (name, ANONYMOUS_NAMESPACE_PREFIX,
1686		ANONYMOUS_NAMESPACE_PREFIX_LEN) == 0)		ANONYMOUS_NAMESPACE_PREFIX_LEN) == 0)
1687		{		{

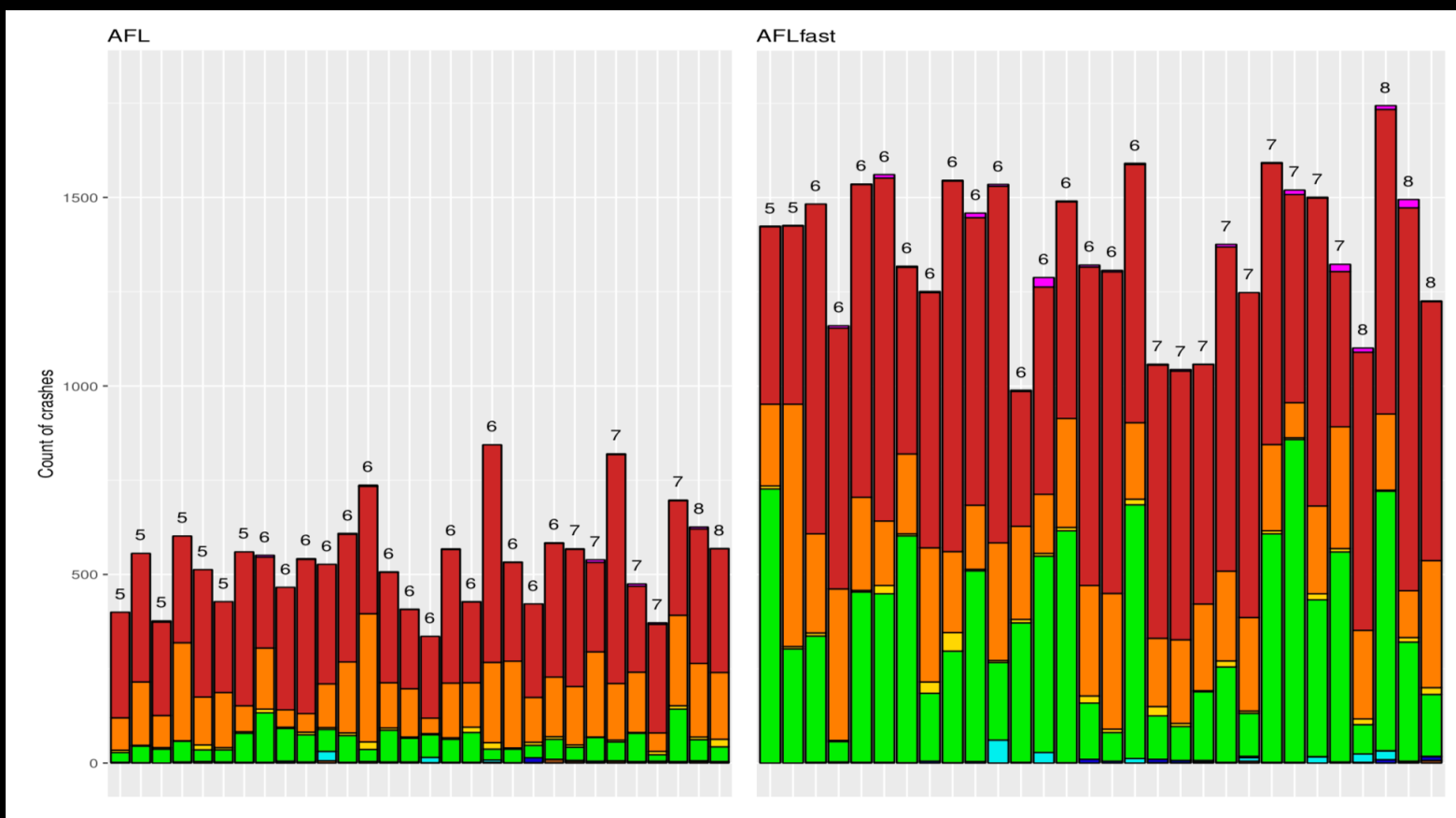
- Used commit history to find **patches since fuzzed version**
 - E.g., commit on left fixes integer overflow
- Applied patches iteratively, and re-ran against all **57,000+ crashing inputs** (post-CMIN, all 30 runs)
 - Those that no longer crash are due to this patch
 - Broke apart patches that fix multiple bugs
- Re-run must account for non-determinism
 - Used ASAN/UBSAN: “non crash” only if it found no issue

Stack Hashes (N=3)

Bug	# Hashes	Matches	False Matches	Input count
A	9	2	7	228
B	362	343	19	31,103
C	24	21	3	106
D	159	119	40	12,672
E	15	4	11	12,118
F	15	1	14	232
G	2	0	2	2
H	1	1	0	568
I	4	4	0	10
unfixed	28	12	16	98
unknown	4	0	4	4

- 57,040 inputs handled by bugfix
 - 98 inputs never fixed
 - 4 inputs “fixed” but due to some source of nondeterminism
- In general: Far less over counting
 - At most 596 hashes for **9 bugs**
 - vs. 57,040 inputs for 9 bugs
- Hashes have false negatives
 - Bug B has 343 hashes that apply just to this bug, but 19 that apply to others too

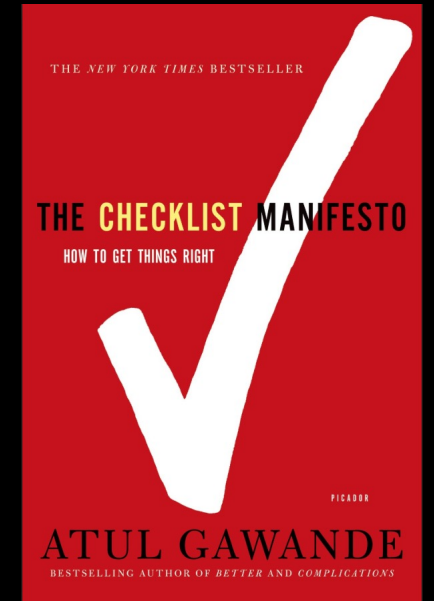
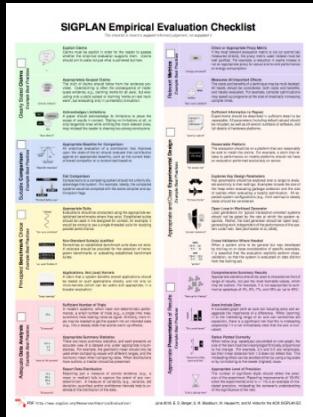
“Unique crashes” (proxy) vs. bugs found



- No trial found more than 8 bugs
- Out of 9 total
- 3 bugs account for most crashing inputs
 - many bugs have few inputs
 - so counting inputs misleading
- Number of crashing inputs correlates with number of bugs, but only loosely
- Mann Whitney p-value is .066 for AFLFast *bugs* > AFL bugs
 - vs. 10^{-10} for “unique” crashes

Influence: Empirical evaluation checklist

- Researchers should know better. But they forget or are nudged away from best practice by culture and circumstance
 - Especially when best practice is more effort
- **Solution:** List of recommendations
 - And identification of open problems
- Inspiration for effort to provide checklist broadly
 - SIGPLAN Empirical Evaluation Guidelines
 - <http://sigplan.org/Resources/EmpiricalEvaluation/>



Influence: FuzzBench

- <https://github.com/google/fuzzbench>

Our paper

Why do we need a fuzzer benchmarking platform?

Evaluating fuzz testing tools properly and rigorously is difficult, and typically needs time and resources that most researchers do not have access to. A study on [Evaluating Fuzz Testing](#) analyzed 32 fuzzing research papers and has [found](#) that *"no paper adheres to a sufficiently high standard of evidence to justify general claims of effectiveness"*. This is a problem because it can lead to [unreproducible](#) results.

We created FuzzBench, so that all researchers and developers can evaluate their tools according to the [best practices](#) and [guidelines](#), with minimal effort and for free.

SIGPLAN Guidelines

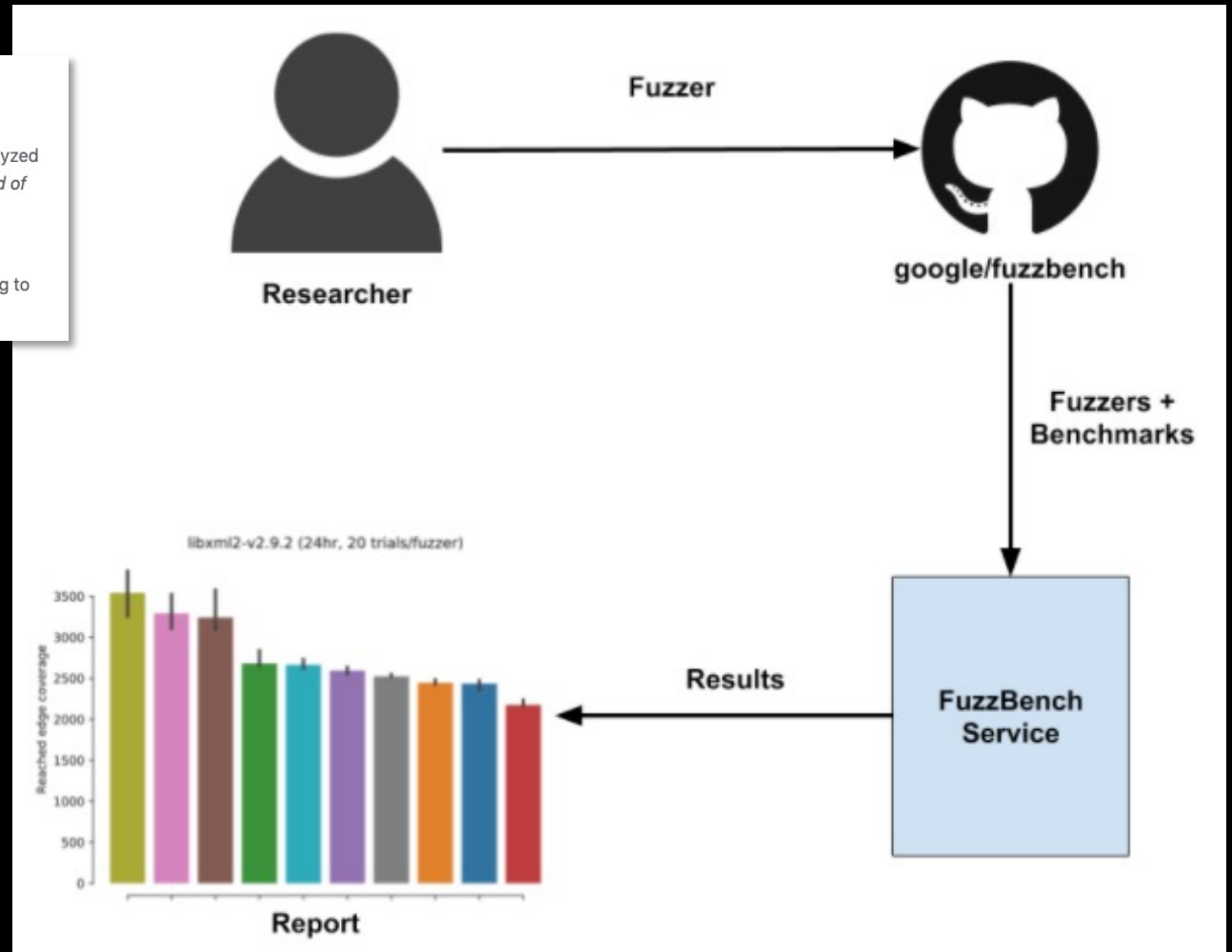
Influence: FuzzBench (coverage driven)

- <https://github.com/google/fuzzbench>

Why do we need a fuzzer benchmarking platform?

Evaluating fuzz testing tools properly and rigorously is difficult, and typically needs time and resources that most researchers do not have access to. A study on [Evaluating Fuzz Testing](#) analyzed 32 fuzzing research papers and has [found](#) that *"no paper adheres to a sufficiently high standard of evidence to justify general claims of effectiveness"*. This is a problem because it can lead to [unreproducible](#) results.

We created FuzzBench, so that all researchers and developers can evaluate their tools according to the [best practices](#) and [guidelines](#), with minimal effort and for free.



Influence: Bug-driven benchmarks

Magma: A Ground-Truth Fuzzing Benchmark

Ahmad Hazimeh
EPFL

Adrian Herrera
Australian National University &
Defence Science and Technology
Group

Mathias Payer
EPFL

ABSTRACT

High scalability and low running costs have made fuzz testing the de facto standard for discovering software bugs. Fuzzing techniques are constantly being improved in a race to build the ultimate bug-finding tool. However, while fuzzing excels at finding bugs in the wild, evaluating and comparing fuzzer performance is challenging due to the lack of metrics and benchmarks. For example, crash count—perhaps the most commonly-used performance metric—is inaccurate due to imperfections in deduplication techniques. Additionally, the lack of a unified set of targets results in ad hoc evaluations that hinder fair comparison.

We tackle these problems by developing *Magma*, a ground-truth fuzzing benchmark that enables uniform fuzzer evaluation and comparison. By introducing *real* bugs into *real* software, *Magma* allows for the realistic evaluation of fuzzers against a broad set of targets. By instrumenting these bugs, *Magma* also enables the collection of bug-centric performance metrics independent of the fuzzer. *Magma* is an open benchmark consisting of seven targets that perform a variety of input manipulations and complex computations, presenting a challenge to state-of-the-art fuzzers.

We evaluate six widely-used mutation-based greybox fuzzers (AFL, AFLFast, AFL++, FAIRFUZZ, MOPT-AFL, and honggfuzz) against *Magma* over 200 000 CPU-hours. Based on the number of bugs, reached, triggered, and detected, we draw conclusions about the fuzzers' exploration and detection capabilities. This provides insight into fuzzer performance evaluation, highlighting the importance of ground truth in performing more accurate and meaningful evaluations.

While these metrics provide some insight into a fuzzer's performance, we argue that they are insufficient for use in fuzzer comparisons. Furthermore, the set of target programs that these metrics are evaluated on can vary wildly across papers, making cross-paper comparisons impossible. The deficiencies of these three metrics are discussed in turn.

Crash counts. The simplest method for evaluating a fuzzer is to count the number of crashes triggered by that fuzzer, and compare this crash count with that achieved by another fuzzer on the same target program. Unfortunately, crash counts often inflate the number of actual bugs in the target program [29]. Moreover, deduplication techniques (e.g., coverage profiles, stack hashes) fail to accurately identify the root cause of these crashes [9, 29].

Bug counts. Identifying a crash's *root cause* is preferable to simply reporting raw crashes, as it avoids the inflation problem inherent in crash counts. Unfortunately, obtaining an accurate *ground-truth* bug count typically requires extensive manual triage, which in turn requires someone with extensive domain expertise and experience [1].

Code-coverage profiles. Due to the difficulty in obtaining ground-truth bug counts, code-coverage profiles are another performance metric commonly used to evaluate and compare fuzzing techniques. Intuitively, covering more code correlates with finding more bugs. However, previous work [29] has shown that there is a weak correlation between coverage-deduplicated crashes and ground-truth bugs, implying that higher coverage does not necessarily indicate better fuzzer effectiveness.

The deficiencies of existing performance metrics calls for a re-think of fuzzer evaluation practices. In particular, the performance metrics used in these evaluations must accurately measure a fuzzer's ability to achieve its main objective: *finding bugs*. Similarly, the tar-

FIXREVERTER: A Realistic Bug Injection Methodology for Benchmarking Fuzz Testing

Zenong Zhang[†], Zach Patterson[†], Michael Hicks[‡], and Shiyi Wei[†]

[†]University of Texas at Dallas

[‡]University of Maryland and Amazon*

Abstract

Fuzz testing is an active area of research with proposed improvements published at a rapid pace. Such proposals are assessed *empirically*: Can they be shown to perform better than the status quo? Such an assessment requires a benchmark of target programs with well-identified, realistic bugs. To ease the construction of such a benchmark, this paper presents FIXREVERTER, a tool that automatically injects realistic bugs in a program. FIXREVERTER takes as input a *bugfix pattern* which contains both code syntax and semantic conditions. Any code site that matches the specified syntax is *undone* if the semantic conditions are satisfied, as checked by static analysis, thus (re)introducing a likely bug. This paper focuses on three bugfix patterns, which we call *conditional-abort*, *conditional-execute*, and *conditional-assign*, based on a study of fixes in a corpus of Common Vulnerabilities and Exposures (CVEs). Using FIXREVERTER we have built REVBUGBENCH, which consists of 10 programs into which we have injected nearly 8,000 bugs; the programs are taken from FuzzBench and Binutils, and represent common targets of fuzzing evaluations. We have integrated REVBUGBENCH into the FuzzBench service, and used it to evaluate five fuzzers. Fuzzing performance varies by fuzzer and program, as desired/expected. Overall, 219 unique bugs were reported, 19% of which were detected by just one fuzzer.

by running it on a set of target programs, comparing its performance against that of one or more baseline fuzzers.

A key question is what performance measure to use. One popular measure, employed by Google's FuzzBench [3], is *code coverage*; if a fuzzer *A* (the improvement) is able to generate tests that execute more distinct lines/branches in a target program than the baseline *B*, then one could argue *A* will find more bugs. Multiple studies have been performed with the goal of understanding the relationship between code coverage and bug finding [4–6]. Unfortunately, a recent study finds that while there is a strong correlation between the coverage achieved and the number of bugs found by a fuzzer, there is not strong agreement on which fuzzer is superior if coverage is used to compare the fuzzers [7].

Another popular measure is to count the number of distinct, crash-inducing inputs generated, a.k.a. *unique crashes*. Since two different inputs can easily trigger the same bug, researchers often employ deduplication heuristics; two popular heuristics are AFL's "coverage profiles" and fuzzy stack hashes [8]. However, a study by Klees et al. [9] showed that both heuristics could still yield many false positives (many "deduplicated" inputs still trigger the same bug) and also some false negatives ("deduplicating" an input can actually remove evidence of a distinct bug). For one program, their study found that a result that appeared to show fuzzer *A* was superior to baseline *B* disappeared when ground truth was used, rather than "unique" crash heuristics.

1. Introduction

Discussion points

- We've discussed incentives many times in this course; how do they apply in this circumstance?
- Do any of these results seem to apply to your native field of study?
- Are there still understudied areas regarding how to evaluate fuzzers?
 - I.e., elements that the paper missed?
- How should we balance a high bar for evaluation, but not squelch promising ideas?
 - Fuzzer usage may vary; how should evaluation accommodate that?
- What are the pitfalls of standardizing evaluation elements?

Readings

SoK: Prudent Evaluation Practices for Fuzzing

Moritz Schloegel¹, Nils Bars¹, Nico Schiller¹, Lukas Bernhard¹, Tobias Scharnowski¹
Addison Crump¹, Arash Ale-Ebrahim¹, Nicolai Bissantz², Marius Muench³, Thorsten Holz¹

¹CISPA Helmholtz Center for Information Security, {first.lastname}@cispa.de

²Ruhr University Bochum, nicolai.bissantz@ruhr-uni-bochum.de

³University of Birmingham, m.muench@bham.ac.uk

Abstract—Fuzzing has proven to be a highly effective approach to uncover software bugs over the past decade. After AFL popularized the groundbreaking concept of lightweight coverage feedback, the field of fuzzing has seen a vast amount of scientific work proposing new techniques, improving methodological aspects of existing strategies, or porting existing methods to new domains. All such work must demonstrate its merit by showing its applicability to a problem, measuring its performance, and often showing its superiority over existing works in a thorough, empirical evaluation. Yet, fuzzing is highly sensitive to its target, environment, and circumstances, e. g., randomness in the testing process. After all, relying on randomness is one of the core principles of fuzzing, governing many aspects of a fuzzer’s behavior. Combined with the often highly difficult to control environment, the *reproducibility* of experiments is a crucial concern and requires a prudent evaluation setup. To address these threats to validity, several works, most notably *Evaluating Fuzz Testing* by Klees et al., have outlined how a carefully designed evaluation setup should be implemented, but it remains unknown to what extent their recommendations have been adopted in practice.

In this work, we systematically analyze the evaluation of 150 fuzzing papers published at the top venues between 2018 and 2023. We study how existing guidelines are implemented and observe potential shortcomings and pitfalls. We

1. Introduction

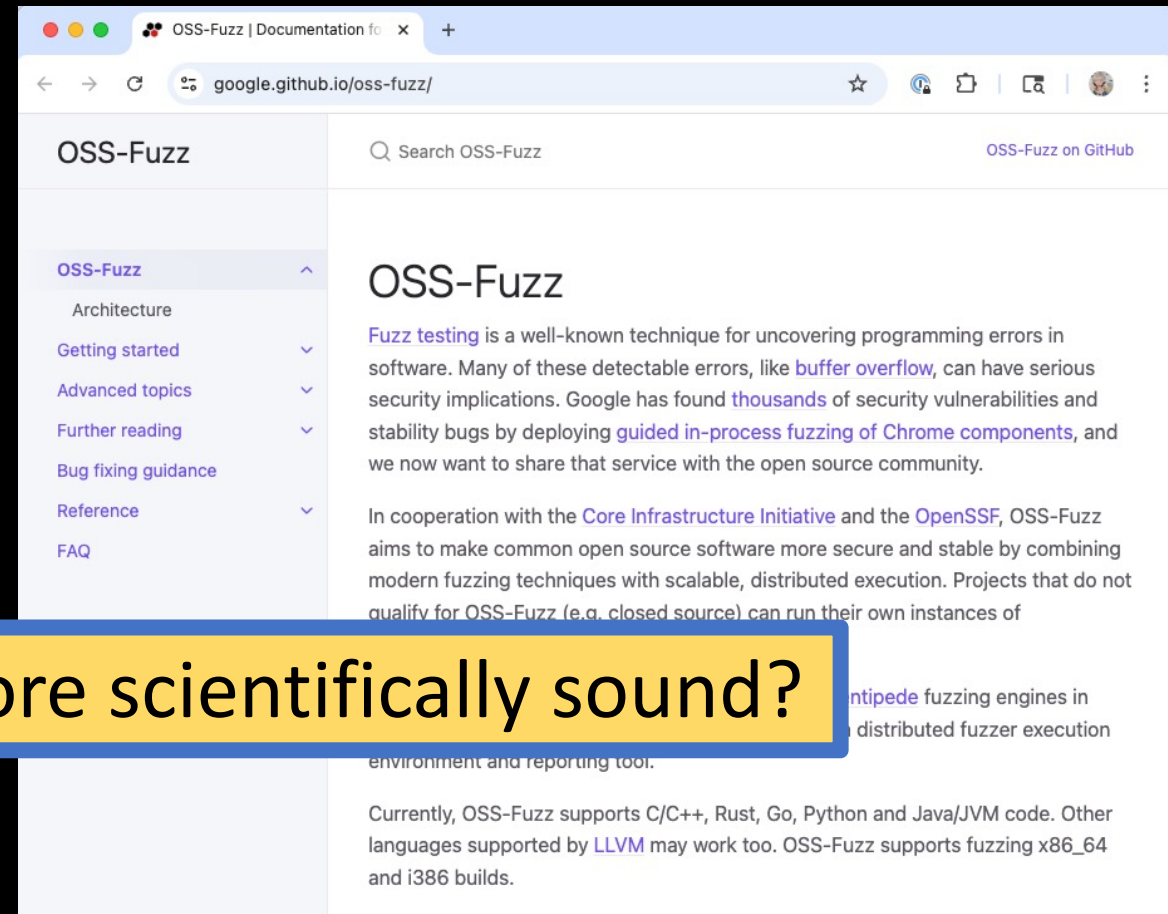
Fuzzing, a portmanteau of “fuzz testing”, has gained much attention in recent years, and the method has proven to be highly successful in uncovering many types of faults in software systems. Companies such as Meta, Google, and Oracle have invested significant resources in this technology and use it to test their products. Large software projects such as web browsers or the Linux kernel incorporate fuzzing into their development cycle, and Google is running an extensive and continuous fuzzing campaign for more than 1,200 open-source projects via OSS-Fuzz [62]. Beyond the wide acceptance in the industry, a large number of academic papers have proposed numerous improvements and novel techniques to enhance fuzzing further. More specifically, we found that, over the past six years, more than 280 papers on fuzzing have been published in the top computer security and software engineering venues.

A cornerstone of fuzzing research, and science in general, is that other researchers can critically assess the correctness of scientific results. To this end, the research results must be *reproducible*, meaning that another group should be able to obtain the same results using the same experimental setup, often by using a research artifact provided by the authors [8]. Reproducibility is paramount for other researchers to understand, trust, and build on the research results.

The 6 years since *Evaluating Fuzz Testing*

- 280 papers on fuzzing published in top security and SE venues
- New benchmarks created (Magma, RevBugBench)
- New papers with evaluation guidelines published

Now running on 1200 OSS packages



The screenshot shows the OSS-Fuzz documentation page on GitHub. The page title is "OSS-Fuzz" and the URL is "google.github.io/oss-fuzz/". The left sidebar contains a navigation menu with links: "OSS-Fuzz", "Architecture", "Getting started", "Advanced topics", "Further reading", "Bug fixing guidance", "Reference", and "FAQ". The main content area has a search bar and the title "OSS-Fuzz". The text describes fuzz testing as a well-known technique for uncovering programming errors in software, mentioning that Google has found thousands of security vulnerabilities and stability bugs by deploying guided in-process fuzzing of Chrome components. It also mentions cooperation with the Core Infrastructure Initiative and the OpenSSF, and states that OSS-Fuzz aims to make common open source software more secure and stable by combining modern fuzzing techniques with scalable, distributed execution. Projects that do not qualify for OSS-Fuzz (e.g. closed source) can run their own instances of OSS-Fuzz. The page also mentions that OSS-Fuzz supports C/C++, Rust, Go, Python and Java/JVM code, and that other languages supported by LLVM may work too. OSS-Fuzz supports fuzzing x86_64 and i386 builds.

On the Reliability of Coverage-Based Fuzzer Benchmarking

Marcel Böhme

László Szekeres

Jonathan Metzman

ABSTRACT

Given a program, we know which fuzzer achieves more coverage as the better one.

Indeed, evaluating 10 fuzzers for 23 hours on 24 programs, we find that a fuzzer that covers more code also finds more bugs. There is a *very strong correlation* between the coverage achieved and the number of bugs found by a fuzzer. Hence, it might seem reasonable to compare fuzzers in terms of coverage achieved, and from that derive empirical claims about a fuzzer's superiority at finding bugs.

Curiously enough, however, we find *no strong agreement* on which fuzzer is superior if we compared multiple fuzzers in terms of coverage achieved instead of the number of bugs found. The fuzzer best at achieving coverage, may not be best at finding bugs.

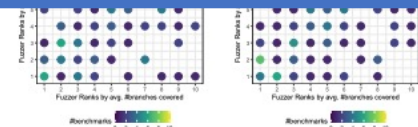
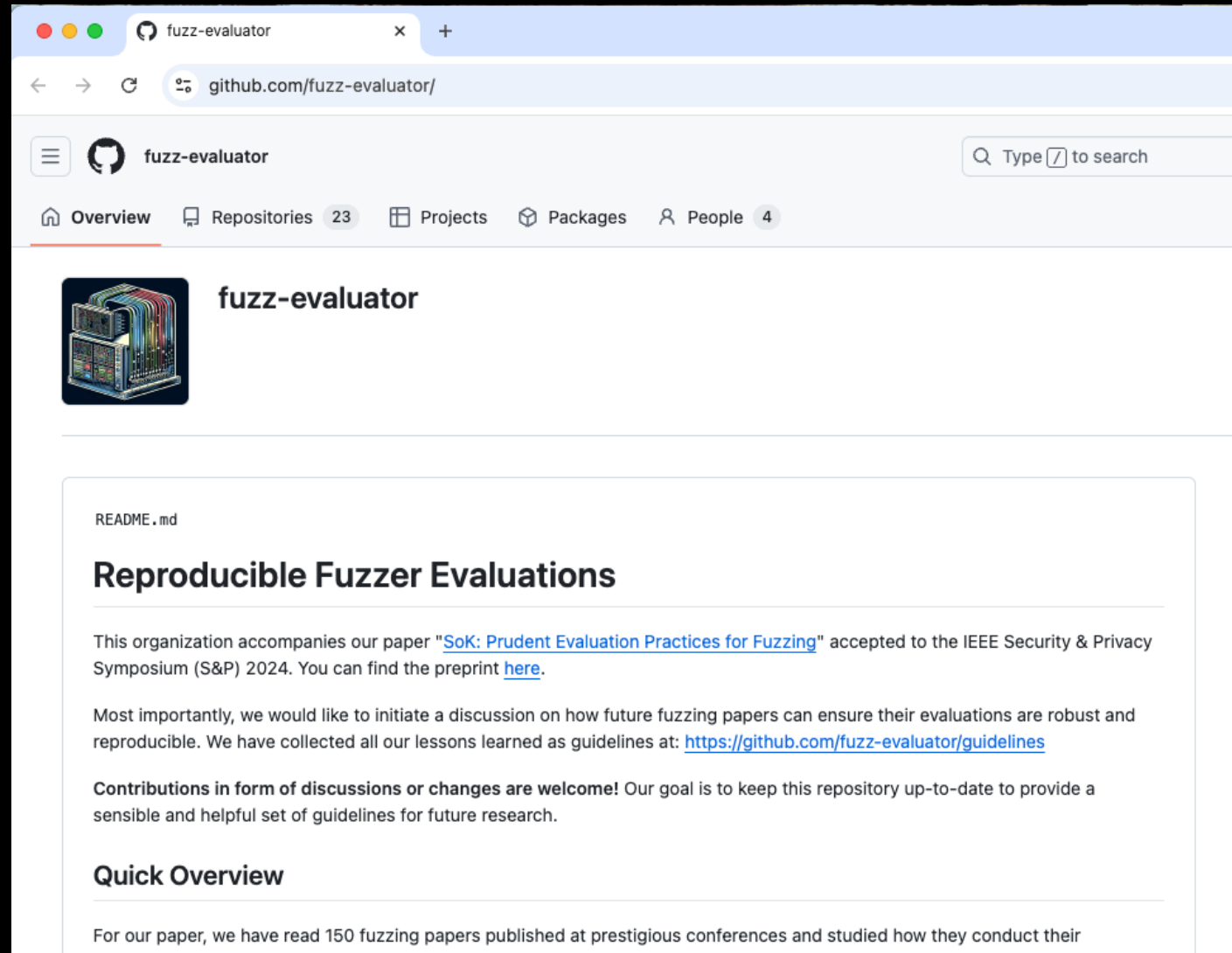


Figure 1: Scatterplot of the ranks of 10 fuzzers applied to 24 programs for (a) 1 hour and (b) 23 hours, when ranking each

Are evaluation practices more scientifically sound?

Method of study

Artifact re-evaluation



fuzz-evaluator

github.com/fuzz-evaluator/

fuzz-evaluator

Overview Repositories 23 Projects Packages People 4

fuzz-evaluator

README.md

Reproducible Fuzzer Evaluations

This organization accompanies our paper "[SoK: Prudent Evaluation Practices for Fuzzing](#)" accepted to the IEEE Security & Privacy Symposium (S&P) 2024. You can find the preprint [here](#).

Most importantly, we would like to initiate a discussion on how future fuzzing papers can ensure their evaluations are robust and reproducible. We have collected all our lessons learned as guidelines at: <https://github.com/fuzz-evaluator/guidelines>

Contributions in form of discussions or changes are welcome! Our goal is to keep this repository up-to-date to provide a sensible and helpful set of guidelines for future research.

Quick Overview

For our paper, we have read 150 fuzzing papers published at prestigious conferences and studied how they conduct their

TABLE 1. OVERVIEW OF ANALYZED PAPERS.

Year	Venue	Papers	Studied
2023	ASE*	[159], [76], [105]	3/7
	FSE*	[166]	1/6
	ICSE	[82], [80], [165], [67], [92]	5/11
	CCS	[182], [47], [32], [116]	4/9
	NDSS	[78], [65], [17]	3/4
	S&P	[108], [19], [104]	3/9
	USENIX	[149], [109], [186], [143], [41], [111], [153], [2], [10], [161], [134], [96]	12/29
2022	ASE	[58], [174]	2/4
	FSE	[66], [189]	2/6
	ICSE	[97], [124], [89], [163], [64], [115], [151], [52]	8/17
	CCS	[83], [12], [57], [144], [37], [29], [191]	7/8
	NDSS	[84], [169], [180]	3/6
	S&P	[74], [147], [102], [28], [100]	5/9
	USENIX	[148], [190], [183], [140], [120], [43], [185], [9], [27]	9/19
2021	ASE	[106], [81]	2/6
	FSE	[118], [181]	2/4
	ICSE	[16], [157], [131]	3/6
	CCS	[61], [192], [122], [54], [72], [33]	6/13
	NDSS	[49], [136], [86]	3/6
	S&P	[117], [40]	2/7
	USENIX	[91], [142], [55], [139]	4/13
2020	ASE	[125], [188]	2/4
	FSE	[22], [152], [145]	3/7
	ICSE	[113], [164], [167], [158]	4/6
	CCS	[171]	1/2
	NDSS	[87], [141], [162]	3/4
	S&P	[132], [6], [168], [75], [48], [34]	6/7
	USENIX	[150], [175], [137], [77], [59], [42], [24], [93], [53], [194], [135]	11/19
2019	ASE	–	0/0
	FSE	[98]	1/4
	ICSE	[36], [126], [39], [172], [160]	5/7
	CCS	[38], [31]	2/3
	NDSS	[69], [5], [7]	3/4
	S&P	[121], [170], [146], [173], [79]	5/5
	USENIX	[68], [35], [187], [85], [13], [110]	6/6
2018	ASE	[95]	1/2
	FSE	–	0/0
	ICSE	–	0/0
	CCS	[25]	1/2
	NDSS	[26], [119]	2/2
	S&P	[60], [133]	2/3
	USENIX	[154], [129], [176]	3/3
total #papers analyzed			150/289

* limited to available preprints

Targets under test

TABLE 2. TARGETS FUZZED IN FIVE OR MORE ANALYZED PAPERS (EXCLUDING BENCHMARKS). SOME PAPERS REPORT GENERICALLY TO EVALUATE ON BINUTILS, WHILE OTHERS SPECIFY EXACT TARGETS, SUCH THAT NUMBERS IN PRACTICE MAY DIFFER SLIGHTLY.

#Uses	Target
25	objdump, readelf
20	nm, tcpdump
19	libpng
17	libtiff
13	cxxfilt, jhead, libjpeg
12	libxml2
11	nasm
10	jasper, libming, openssl, size
9	file, ImageMagick, mjs, tiff2pdf
8	djpeg, exiv2, JavaScriptCore, libarchive, SQLite, v8, xmllint
7	ChakraCore, ffmpeg, harfbuzz
6	binutils, lcms, lrzip, mupdf, OpenJPEG, SpiderMonkey
5	bento, bsdtar, catdoc, cflow, curl, freetype2, GraphicMagick, json, pcre2, proj4, strip, tiff2ps, yara, zlib

Real-world targets are often limited to binary input-affine programs, while benchmarks are not used by the majority of papers. Benchmarks with artificial vulnerabilities are still used.

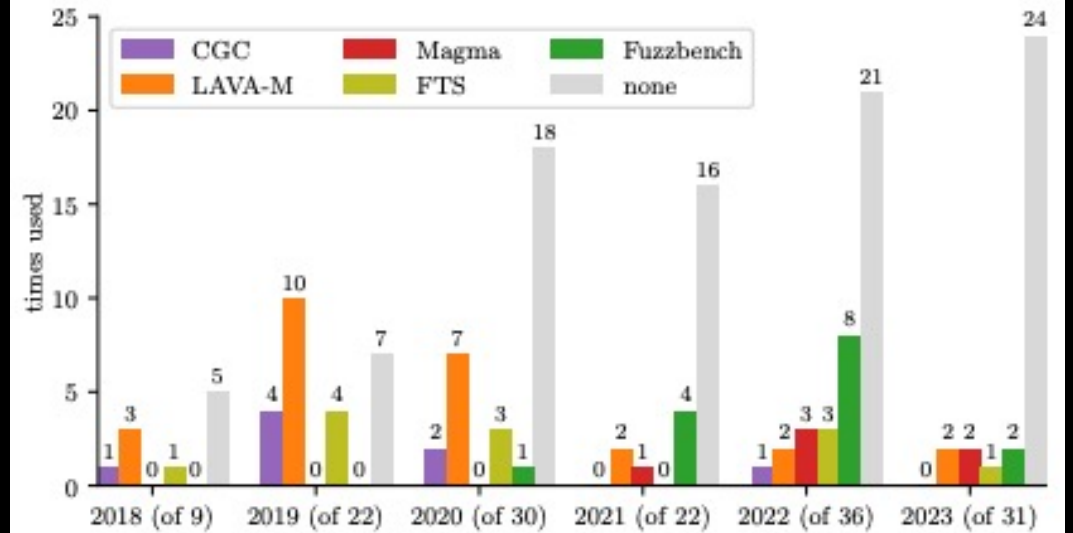


Figure 1. Benchmark usage over the years. The numbers in brackets represent the number of papers analyzed for the respective year. Note that some papers use multiple benchmarks, hence the numbers do not add up.

Trial running times, counts

24

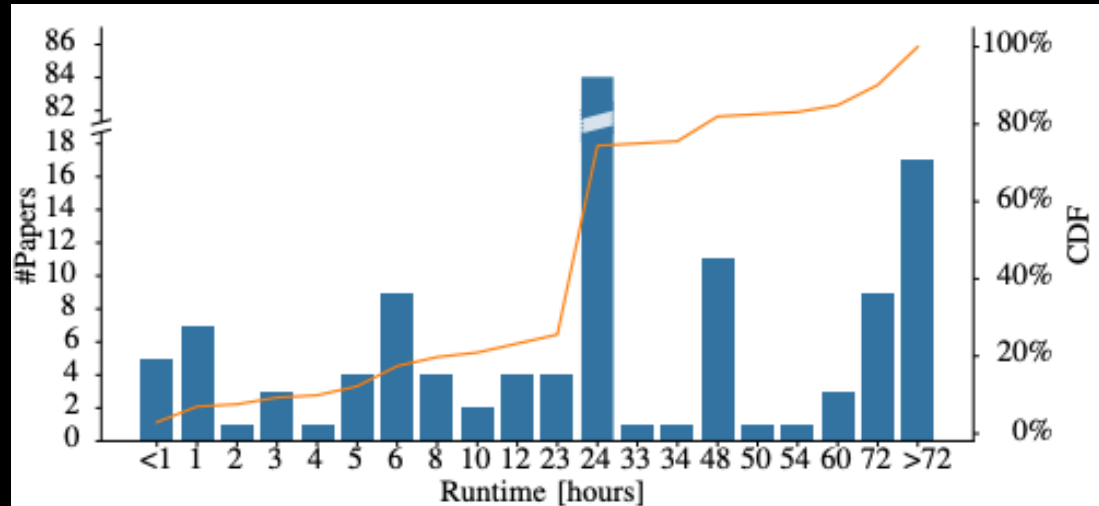


Figure 2. Distribution of runtimes used in practice and cumulative distribution function (CDF), which shows that 27% of papers use a runtime of less than 23 hours. 26 papers use multiple, different runtimes; we include all in these cases.

Running times

5

10

20

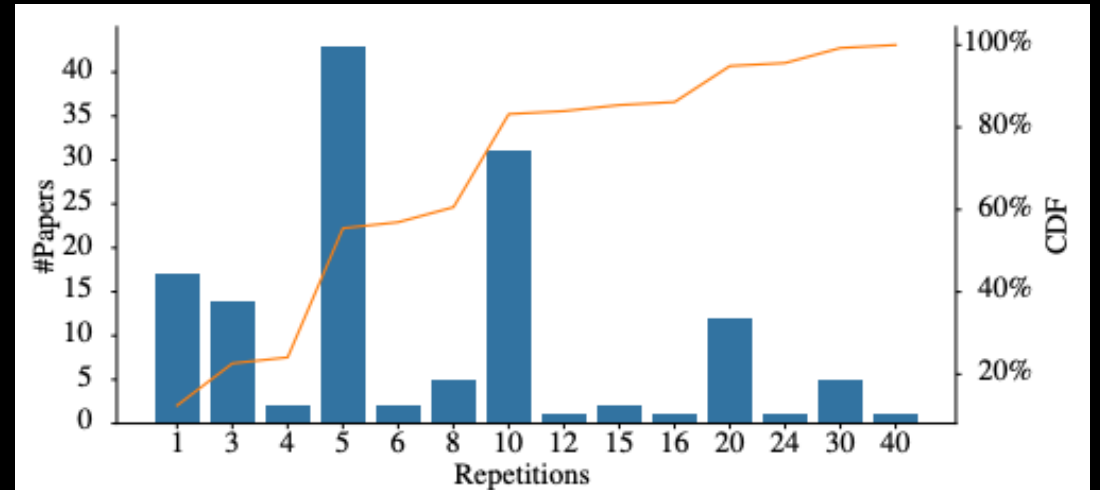


Figure 4. Distribution of trials used in practice and cumulative distribution function (CDF). 8 papers use a different number of trials for different experiments; we include all numbers in this case. Further 21 papers fail to specify the number of trials.

Trials

Measure

- Code coverage
 - Beware: many kinds!
- Known bugs
 - “Known bugs are a good way of measuring a fuzzer’s performance, yet it is difficult to find suitable bugs outside well-designed benchmarks, such as Magma [70] or RevBugBench [183].” – so ... use those?
- Unknown bugs – CVEs

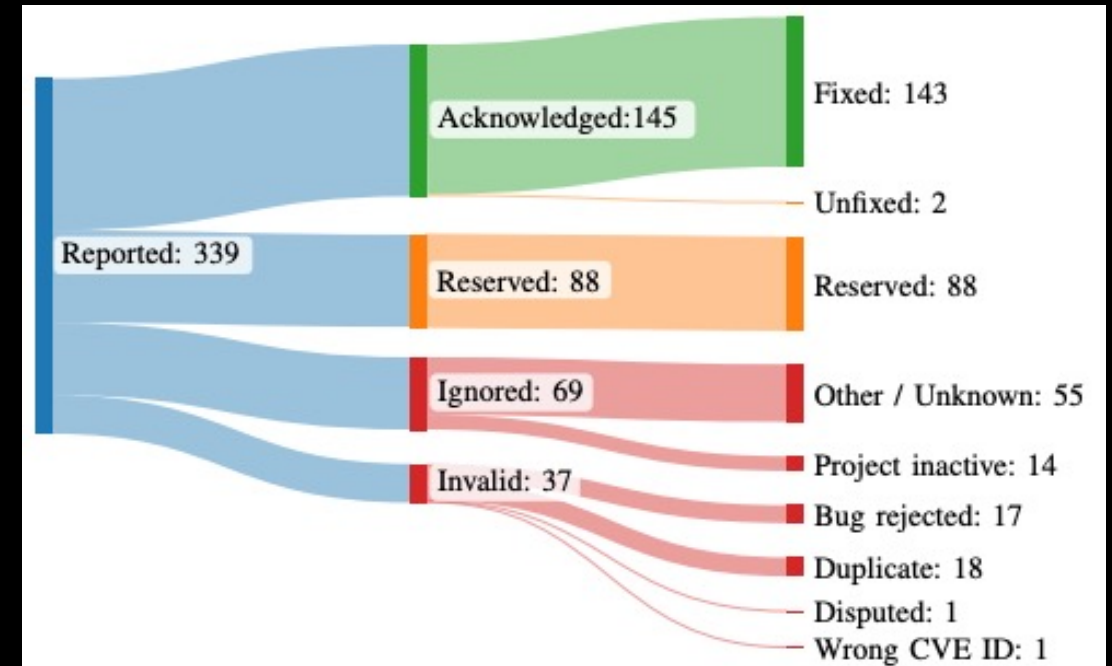


Figure 3. Outcome of 339 CVEs that were reported across 35 papers. Only 43% of the CVEs have been acknowledged by the developers. Pending public disclosure, information on CVEs in the *Reserved* state is withhold.

Statistical analysis

- 63% (94) conduct no statistical test of significance of new fuzzer vs. a baseline's performance
- 37% (55) of the papers run a Mann-Whitney U test to measure statistical significance
 - 15% (22) of the analyzed papers conduct a Mann-Whitney U-test while having five or fewer trials. Insufficient # trials risks that it may never reject the hypothesis (captured in higher p value).
- 73% (109) of the papers provide no measure of uncertainty
- 12% (18) present a measure like effect size (like A12).

Artifacts evaluated: Reproducibility problems

- Statistically sound comparison diminishes published results
- Unequal experimental setup between new fuzzer and baseline led to inflated results for new fuzzer
 - Seems likely unintentional, but also necessary for publication
- Lack of standard metrics means new ones lack context
 - Do the new ones correlate with finding bugs? You can measure that ...

Revisited

Fuzzing Evaluation Recipe (for new fuzzer A)

- A compelling **baseline** fuzzer B to compare against ✓ +ensure fair comparison
 - A **sample of target programs** (benchmark suite)
 - Representative of larger population ✓ +prefer standards
 - A **performance metric**
 - Ideally, the number of bugs found (else a proxy) ✓ +standard code coverage
beware CVEs
 - A meaningful set of configuration parameters ✓
 - Notably, justifiable **seed file(s)**, **timeout**
 - A sufficient **number of trials** to judge performance ✓ or bootstrap tests, ANOVA
 - Comparison with baseline using a **statistical test** +report uncertainty, effect size
- ✓ And: Publish a reproducible artifact

Discussion points

- What different phenomena did this paper find compared to the 2018 paper?
- What areas warrant further investigation?
- This paper focuses more on reproducibility of artifacts – are there implications for artifact evaluation in review processes
- This paper essentially ratified the 2018 paper's evaluation advice. Given how research has been conducted, do you think further steps should be taken?